T&RH-MONITORING · PROPOSAL FOR REVIEW

TCP/IP Instruments

How a Modbus TCP/IP device becomes an instrument in the application — the flow, where it runs, and every screen that changes.

Draft · 31-10-2026 | Concept only — nothing is built yet | Controlytics

1
choice on the profile
Gateway (today's process) or TCP/IP

1
address and key list per instrument
its own IP, port and register map

1
path for every reading
limits, alarms, audit, reports — unchanged

THE ANSWER IN SHORT

The application does the reading itself. A small part of the application — the **TCP/IP Reader** — connects to each TCP/IP instrument's IP address and port, reads its registers every 60 seconds, and turns them into that instrument's own keys. The desktop program in `TCPIP/` is no longer needed.

Each instrument holds its own IP address, port and register map — so instruments in one profile can carry different keys. **The profile holds only how they are read** (every 60 s, unit ID 1) and which key names are judged. Every reading then enters the same path MQTT, HTTP and gateway readings use today, so offsets, limits, alarms, history, dashboards, reports and the audit trail work without any change.

Gateway profiles do not change at all.

NEW new on this screen | SAME as today | 1 numbered note beside each screen

CONTENTS

1 The whole flow — set up once, then read every minute 2
2 Where the Reader runs — the one decision 3
3 Screen · New profile: Gateway or TCP/IP 4
4 Screen · New instrument: its own IP, port and keys 5
5 Screen · Test connection before it is saved 6
6 Screen · Instrument page: connection status 7
7 Screen · Plant Readers (only if the server is in AWS) 8
8 Rules, and what stays the same 9
9 Decisions needed from you 10

1 THE FLOW

Set up once, then read every minute

SET UP ONCE — a Super Admin, on screens that exist today plus the new parts

STEP 1 · PROFILES

New profile → choose the connection

Gateway — the device sends its own data (MQTT, HTTP, TCP, or through a gateway). **Today's process.**
TCP/IP — the application reads each device; set how often (60 s) and the unit ID, once.

→

STEP 2 · NEW INSTRUMENT

Its own IP · Port · keys

`192.168.1.24 : 502`

Temperature	40001 · FLOAT32
Humidity	40003 · FLOAT32
Pressure	40005 · FLOAT32
Voltage	40009 · UINT16

→

STEP 3 · BEFORE SAVING

Test connection

Reads the device once with **this instrument's** register map and shows every key's value **before** the instrument is saved. Nothing is stored.

→

STEP 4 · SAME AS TODAY

Limits, offsets, location, attributes

Set exactly as for every instrument today. A TCP/IP instrument gets **no access token** — nothing to leak.

EVERY 60 SECONDS, AUTOMATICALLY — for each enabled TCP/IP instrument

PLANT NETWORK

Modbus TCP/IP device

`192.168.1.24 : 502`

sensor, PLC or energy meter. Holds its values in registers 40001…

←

NEW · INSIDE THE APPLICATION

TCP/IP Reader

Connects, reads **40001–40010 in one request** (function 03), decodes each key by **that instrument's** register map.

Temperature	**22.4**
Humidity	**47.1**
Pressure	**101.7**

→

SAME AS TODAY · ONE PATH FOR EVERY READING

Ingest

offsets · limits judged · alarms raised · latest value · history · dead letters

Temperature and Humidity are **judged** (the profile's measurements). Pressure, Power, Voltage, Switch are **shown, never judged**.

→

SAME AS TODAY

Everywhere a reading shows

Dashboard & built dashboards · Trend page · Data Report · Alarms · Settings · Audit trail

GATEWAY PROFILES — SAME

Device or gateway pushes

MQTT · HTTP · WebSocket · raw TCP, with the instrument's own token

→

The same Ingest

Both kinds of instrument end up identical: same tables, same alarms, same reports.

ONE READ CYCLE — what happens when it fails

connect → read 10 registers → decode → store reading → wait 60 s

no answer / refused → nothing stored → try again next cycle

silent 15 min → Inactivity alarm — the existing rule, no change

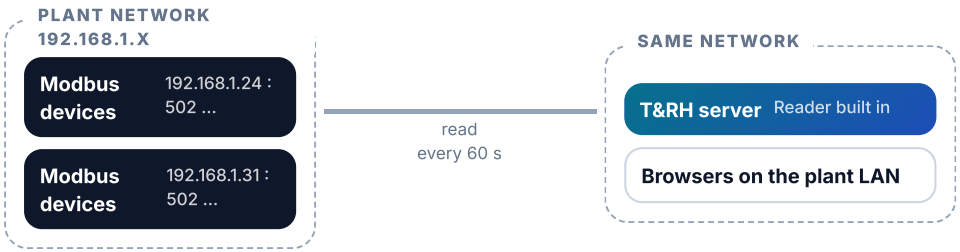
WHERE THE READER RUNS

2 The Reader must sit on the same network as the devices

Modbus devices live on the plant network (`192.168.1.x`). Something on that network has to do the reading. The Reader is built **once** and can run in either of two places — the screens and the data are identical in both.

A · Server at the plant

Windows Setup.exe or Ubuntu .deb install

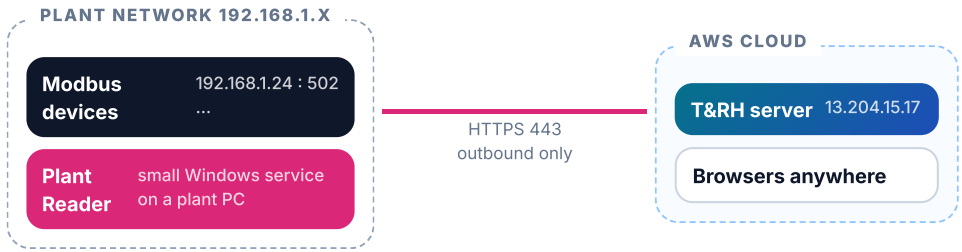


- ✓ Nothing extra to install — the Reader starts with the server
- ✓ Works with no internet at all (intranet only)
- ✓ Simplest to validate: one system, one audit trail
- ! Needs the server on, or routed to, the devices' network

What is installed	**Only the T&RH server, as today**
Who reads the devices	**The server itself**
Network ports	**Server → device TCP 502, inside the plant**
Internet needed	**No**
If the server stops	**Nothing is read; Inactivity alarms when it returns**

B · Server in AWS

like today's https://13.204.15.17



- ✓ The plant PC only **calls out** — no port opened into the plant
- ✓ It downloads its instrument list and register maps from the application — nothing is typed on the PC
- ✓ Readings go up with the Reader's own key, revocable on one screen (page 8)
- ! One more thing to install and keep running on a plant PC (it replaces today's desktop tool)

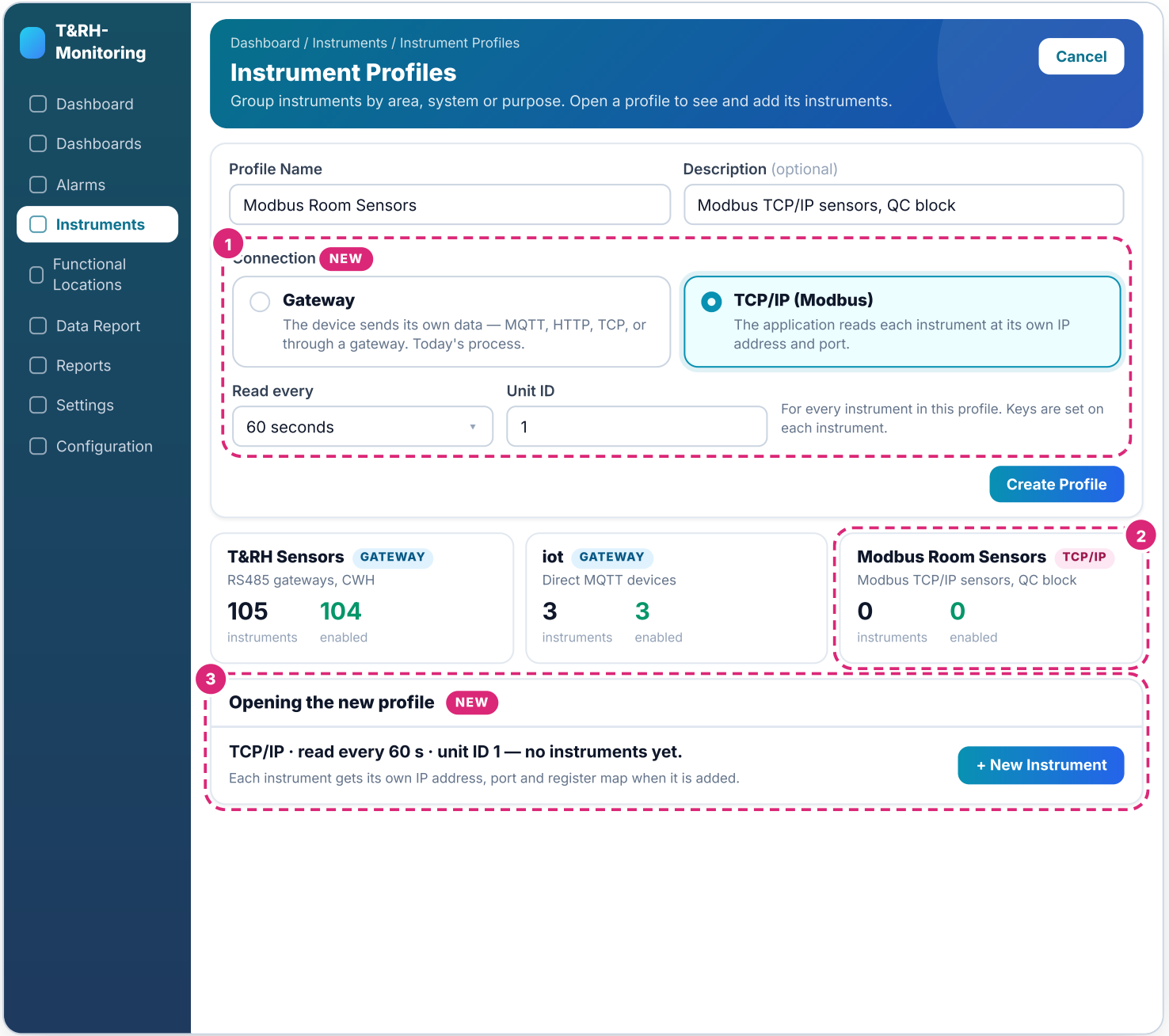
What is installed	**Plant Reader service on one PC per plant network**
Who reads the devices	**The Plant Reader**
Network ports	**PC → device TCP 502; PC → AWS HTTPS 443 out**
Internet needed	**Yes, outbound only**
If the internet drops	**Readings are kept on the PC and sent when it returns (page 8)**

Your decision → Will production run at the plant (A), in AWS (B), or both? My recommendation: build the Reader once and offer both — A first if Granules' production server is on-premises, B first if it stays in AWS.

3

SCREEN · INSTRUMENTS → INSTRUMENT PROFILES → + NEW PROFILE

New profile: choose Gateway or TCP/IP



WHAT CHANGES

1. **One new choice when a profile is created.** Gateway is selected by default, so nothing changes for anyone who ignores it.
2. **Every profile card shows its connection** — so a list of profiles tells you at a glance which ones are read by the application.
3. **The profile holds only how its instruments are read** — every 60 s, unit ID 1. It has no keys of its own: each instrument gets its own register map (next page).

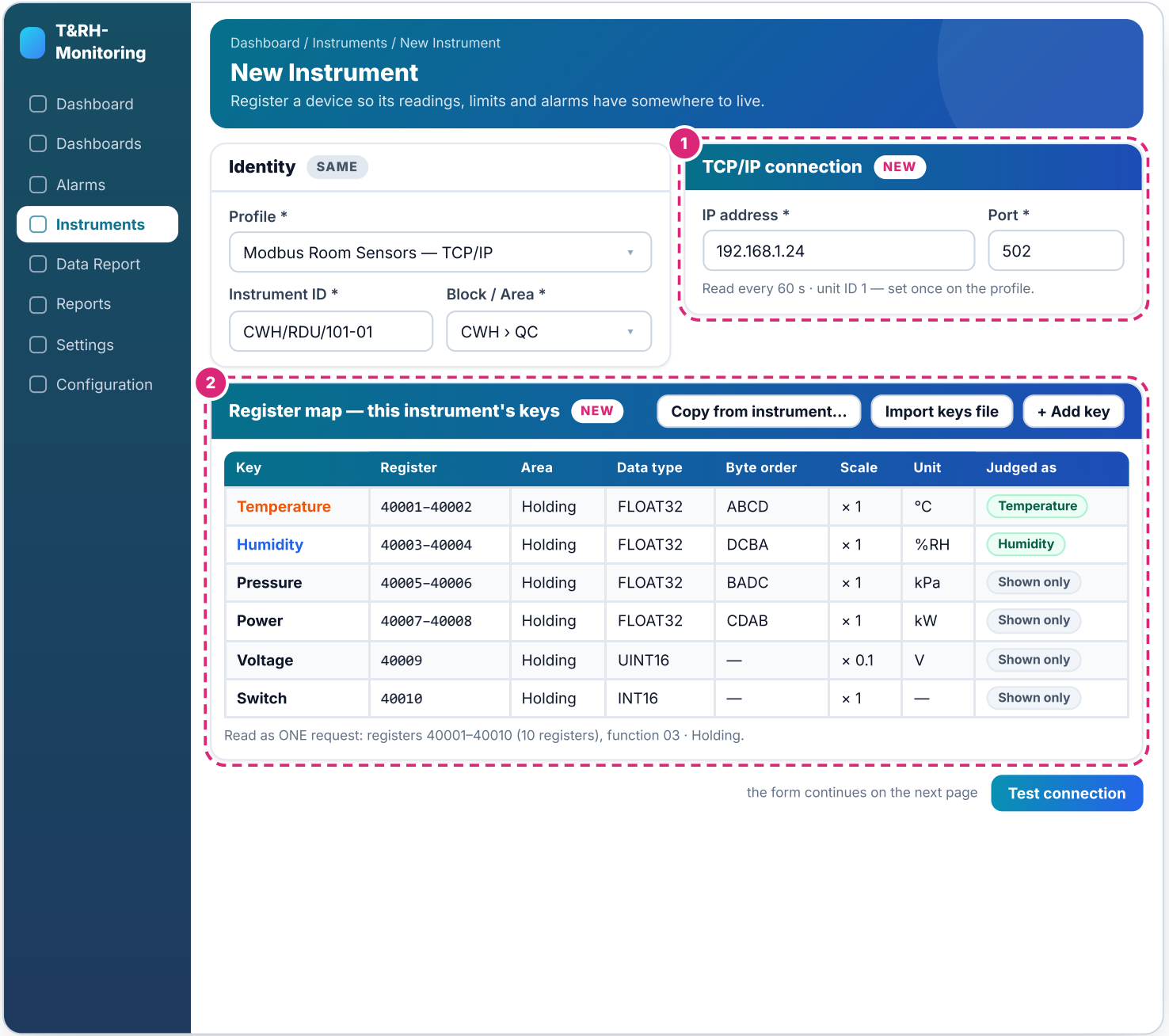
Why "TCP/IP (Modbus)" and not just "TCP/IP"? The application already accepts devices that *push* over raw TCP (a Gateway-type connection). The word Modbus keeps the two apart.

Can it be changed later? Only while the profile has no instruments — afterwards the two kinds of instrument hold different settings.

Existing profiles (T&RH Sensors, iot) become Gateway automatically. No data moves.

SCREEN · INSTRUMENTS → NEW INSTRUMENT (PROFILE IS TCP/IP)

4 New instrument: its own address and its own keys



WHAT CHANGES

1. **Its own IP address and port.** Shown only when the chosen profile is TCP/IP; port 502 is filled in. "Is Gateway" and "Reports through gateway" are not offered.
2. **Its own register map.** Instruments in one profile can carry different keys — 101-01 sends six; the next device may send only Temperature and Pressure.

Identical devices are not typed twice. Copy from instrument... takes another instrument's map; **Import keys file** reads today's `ModbusTCPReader_keys.json`.

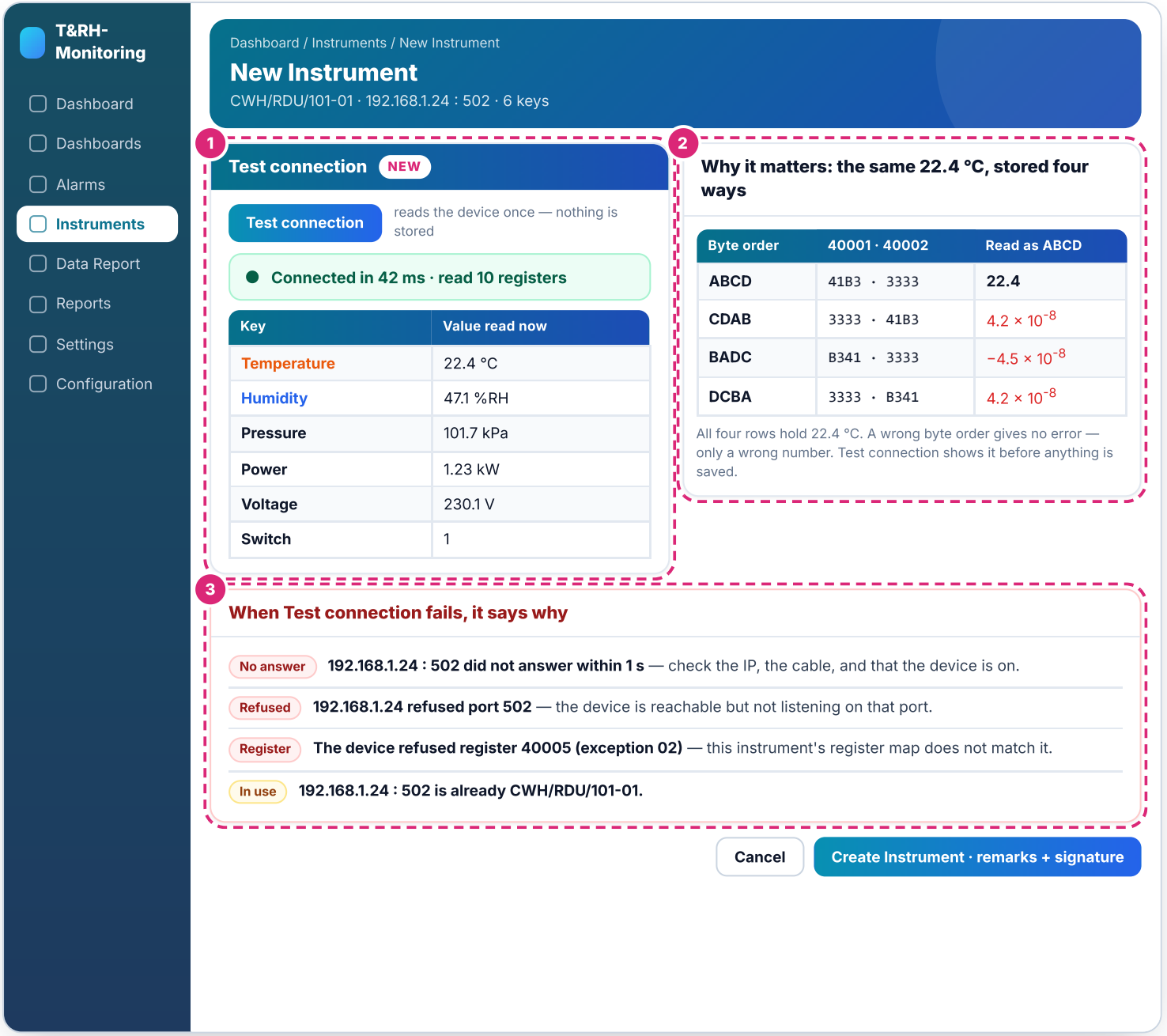
Judged as still comes from the profile's Measurements: a key named Temperature or Humidity (device keys temperature, t · humidity, h) gets limits and alarms; every other key is shown, never judged.

Checked before saving: two keys in one register, a FLOAT32 running past the map, more than 125 registers in one read. The server refuses the same.

The cost of keys per instrument: when a device model changes, each of its instruments is edited — quick with Copy from instrument, and every change is in the audit trail.

SCREEN · NEW INSTRUMENT, CONTINUED

5 Test connection: every key's value before it is saved



WHAT CHANGES

1. **Test connection** reads the device once with **this instrument's** register map and shows every key's value, so a wrong IP, register or byte order is caught **before** the instrument exists. Nothing is stored.
2. **Why it is there:** a wrong byte order does not fail, it reads a believable-looking wrong number — or a tiny one, as here.
3. **Every failure is named in plain words.** The instrument can still be saved (a device may be installed later); it shows "not answering" until it does.

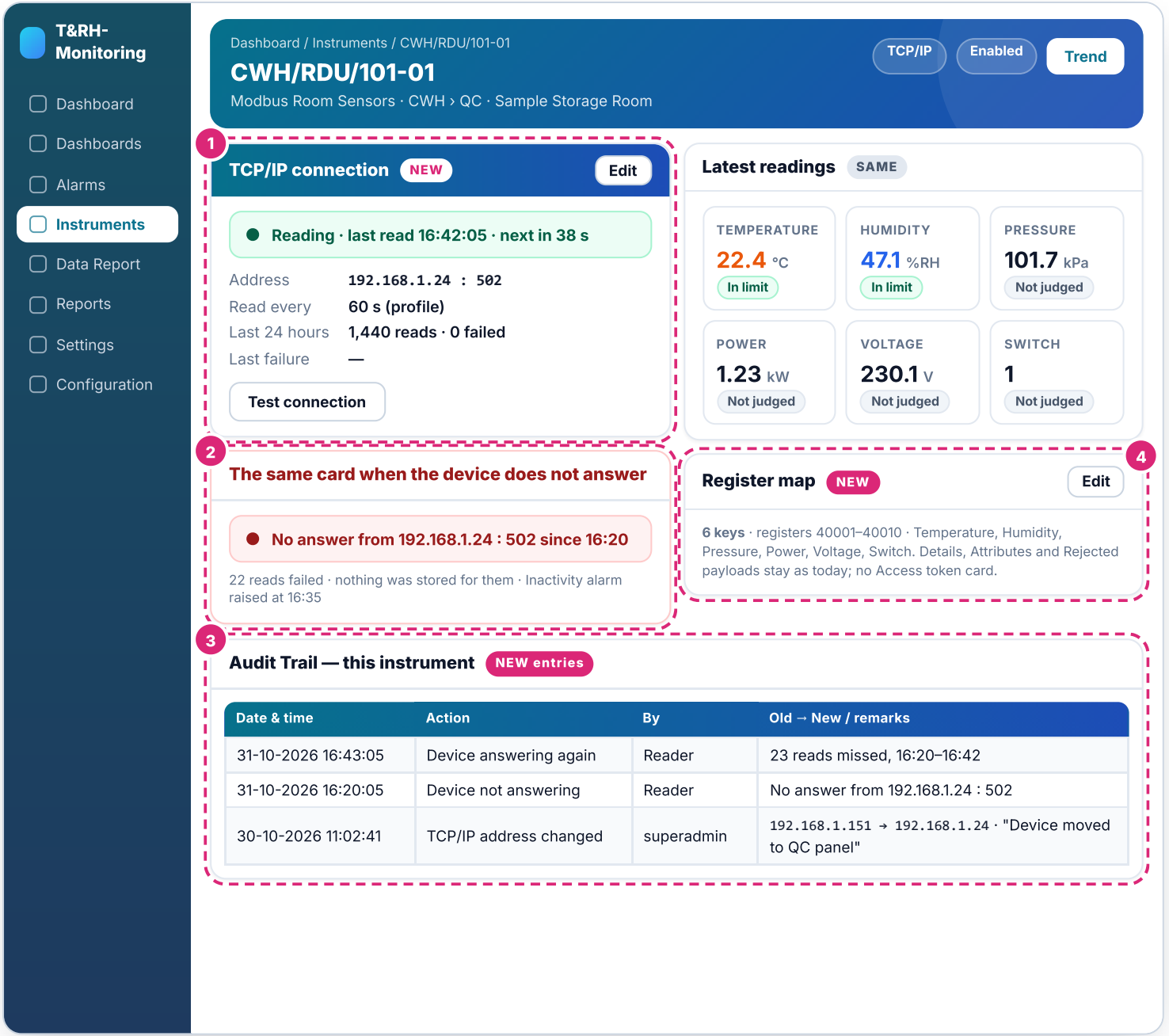
One address = one instrument. `192.168.1.24 : 502` on a second instrument is refused, naming the one that has it.

No access token for a TCP/IP instrument — the device never logs in; the application reads it.

Attributes (limits, offsets, Location) are entered as for every instrument today. **Bulk upload** gets IP address and Port columns; each register map is copied or imported.

SCREEN · INSTRUMENTS → CWH/RDU/101-01

6 Instrument page: is the application reaching it?



WHAT CHANGES

1. **One card says whether the application is reaching the device**, when it last read it and how many reads failed. **Edit** changes the address with remarks and signature, recorded in the audit trail.
2. **When it fails, it says why in plain words** — no answer, connection refused, or the device refused a register — and stores nothing. After 15 silent minutes the existing **Inactivity alarm** is raised, exactly as for a gateway sensor today.
3. **The audit trail explains every gap.** An address change carries the old and new value and its remarks; the Reader records when a device stopped and started answering, so a missing hour in a report has a recorded reason.
4. **The instrument's own register map**, editable here with remarks and signature; the old and new map are in the audit trail.

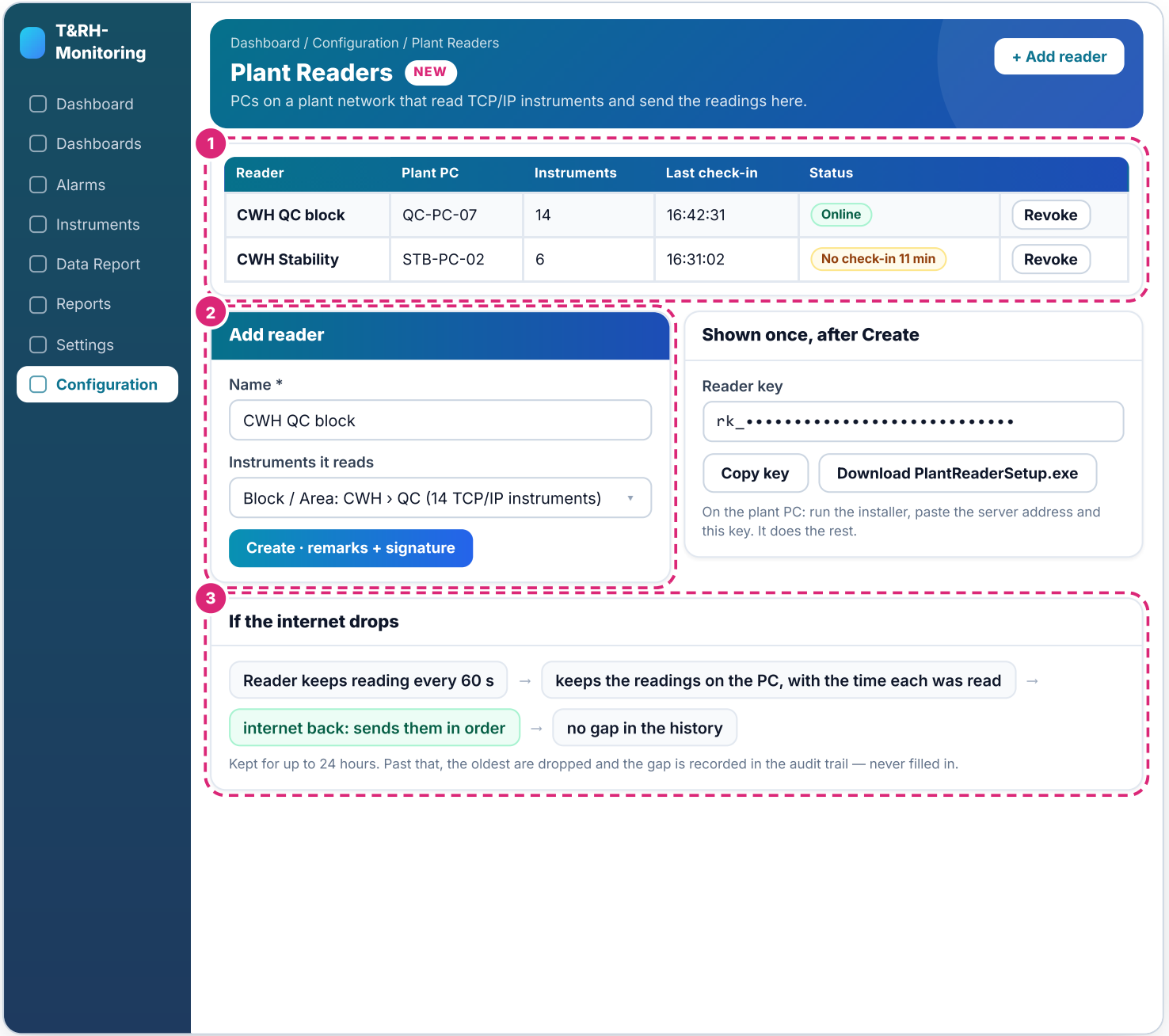
Everything else on this page, the dashboards, the trend page, the Data Report and the alarms page is unchanged — a TCP/IP reading is an ordinary reading once it is stored.

Dashboards: Pressure, Power, Voltage and Switch can be put on the new gauges, bars and charts straight away — they are the "different keys per profile" case phase 2 was built for.

7

SCREEN · CONFIGURATION → PLANT READERS · ONLY FOR OPTION B (SERVER IN AWS)

Plant Readers: the PCs that read devices for a cloud server



WHAT CHANGES

1. **Every plant PC that reads devices is listed**, with the instruments it covers and when it last checked in. A reader that stops checking in is visible here, and its instruments raise the normal Inactivity alarms.
2. **Adding a reader gives a key, shown once.** The plant PC uses it to fetch its instrument list and register maps and to send readings — over HTTPS, outbound only. **Revoke key** cuts it off at once.
3. **An internet outage does not lose readings.** The Plant Reader holds them and sends them late, each with its real read time — so the trend and the Data Report have no hole. **One point to settle in the design:** today a late reading is stored but not judged, so an excursion during the outage would not raise an alarm when it arrives.

Not needed for option A. When the server is at the plant, the Reader runs inside it and this page does not appear.

Replaces the desktop tool: no settings are typed on the plant PC, and no device password is stored in a file there.

RULES

8 What the feature promises — and what stays exactly as it is

SAME Stays exactly as it is

- **Gateway profiles and instruments** — MQTT, HTTP, WebSocket, raw TCP, gateways, tokens, device certificates
- **Limits, offsets, decimals** — set per instrument, as today
- **All five alarm sources** — limits, inactivity, RS485, sensor range, offline
- **Dashboard, built dashboards, trend page, Data Report, Alarms, Settings**
- **Report review, downloads and signatures**
- **Audit trail** — and every new setting is recorded in it

NEW How TCP/IP instruments behave

- **Each instrument has its own register map** — keys can differ between instruments in one profile
- **Read-only.** The application never writes to a device — writing to plant equipment from a 21 CFR Part 11 system is a separate decision
- **One path:** every reading enters the same ingest as MQTT readings — no second way into the database
- **Each instrument has its own IP and port** — no two instruments share one
- **Every 60 s by default**, per profile, never faster than 10 s
- **A reading's time is the moment it was read**
- **A failed read stores nothing** — never a zero, never a repeat of the last value
- **Disabled instrument = not read**

21 CFR Part 11 and validation

- **Register map, connection and address changes** need remarks and a signature, like every instrument change today
- **Each change is an audit record** with the old and new value
- **Test connection** reads only and is not stored — it cannot create a reading
- **The Reader's own events** (started, device not answering, recovered) are logged with times, so an outage can be explained later
- **OQ test cases** to add: register map, test connection, a failed device, an address clash, a disabled instrument

HOW IT WOULD BE DELIVERED — each part tested and usable on its own

Part 1 Profile, instrument address and keys

The Gateway / TCP/IP choice, IP, port and register map on each instrument (Copy from instrument, Import keys file), Test connection, the audit records. Nothing is read automatically yet.

Part 2 The Reader inside the server (option A)

Reads every enabled TCP/IP instrument on its profile's schedule, through the same ingest; the connection card on the instrument page; device-not-answering records.

Part 3 Plant Reader for AWS (option B)

The Windows service, the Plant Readers page and keys, keeping readings through an internet outage; bulk-upload columns; OQ test scripts for all of it.

TODAY'S 101-01 AND THE DESKTOP TOOL

Create the TCP/IP profile → Move 101-01 to it: enter 192.168.1.24 : 502, import the keys file → Test connection → readings arrive from the application → Close the desktop tool →

Withdraw the old 101-01 token

9 YOUR DECISIONS

Five answers, then I write the design and build it

1 Where will the Reader run?

A — inside the server at the plant · B — a Plant Reader on a plant PC, server in AWS · or both. This decides whether page 8 is built.

RECOMMENDED
Build once, offer both. Start with the one Granules' production uses (still an open point in the DS document).

2 What is the option called on screen?

"TCP/IP" alone could be confused with the raw-TCP push devices already use.

RECOMMENDED
"TCP/IP (Modbus)".

3 How often is a device read?

The desktop tool reads every 200 ms — 2.6 million rows a day for one instrument. The plant reports per minute today.

RECOMMENDED
60 seconds by default, set per profile, never under 10 seconds.

4 Should the application ever write to a device?

The desktop tool can write registers and coils. In a validated system that is a control action, not monitoring.

RECOMMENDED
No — read-only. Writing would be its own project with its own validation.

5 101-01 and the desktop tool

101-01 is fed today by the desktop tool over unencrypted MQTT, and its token sits in a plain file.

RECOMMENDED
Move 101-01 to the new profile, retire the tool, withdraw the old token (page 9).

WHAT HAPPENS AFTER YOU ANSWER

You answer the five points → I write the design document (reviewed before you see it) → You approve it → Plan, then build Part 1 → 2 → 3 with tests and reviews → You try it on your local application → Deploy when you say